Automating the Management of Software Systems

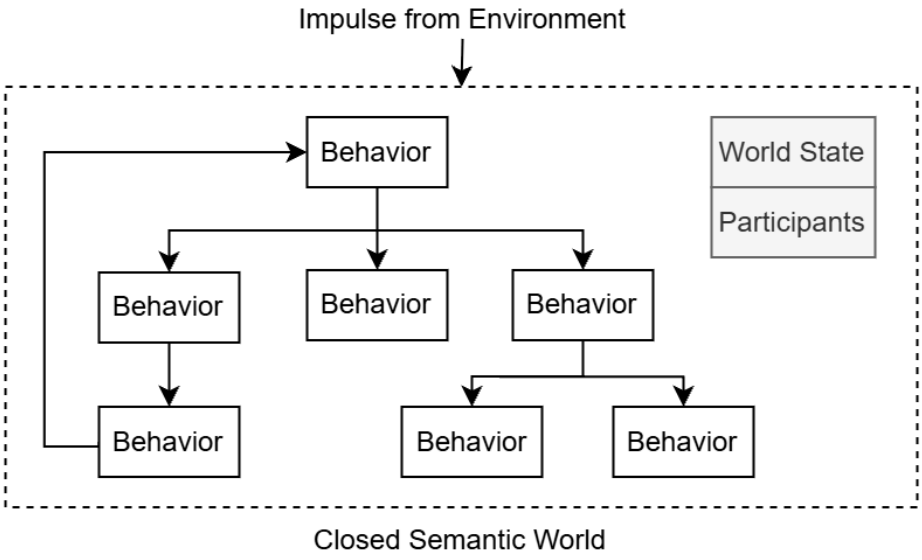
DRAFT

Software systems are an essential part of modern society and, when they fail to function as intended, the implications can be enormous. It is therefore vital that systems are built to minimize the possibility of failure and that, when failures do occur, effective diagnostic processes expedite the systems recovery.

To address this need, in this blog, I will present a framework called the Design Learning Platform (DLP) that fully automates the diagnosis of software failures. First, I will first explore the nature of the design on its own and explain how a closed semantic world can predict and resolve failures. Next, I will talk about how the design is applied to software systems and explain the diagnostic automation that it enables. Finally, I will talk about how this framework enables the complete automation of software system management and discuss how this is practically achieved by the platform.

The Design

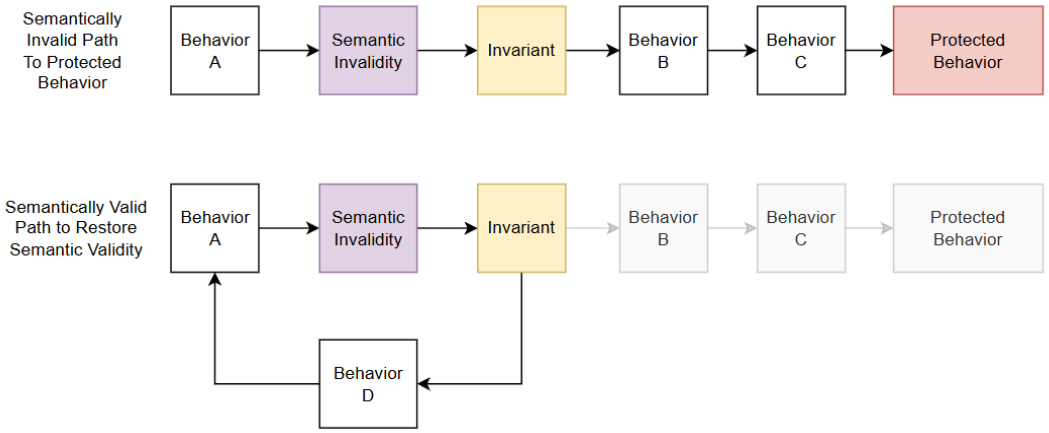
A design is a closed semantic world. It contains participants whose state is transformed through the design's unambiguous behaviors. When the environment interacts with the design, it introduces a change to the state of the world. This state transition may enable new behaviors, which the design deterministically resolves in response to the updated state, producing further state changes. When resolving a state, the design may either exhibit behavior directly or deterministically select which behaviors are enabled as a reaction to the state. The design continues reacting to each resulting state transition until no further behavior is enabled and the world reaches a stable semantic state. This chain of resolutions constitutes the structure of the design itself and provides the environment a fully specified path through it.



The design is said to be in a semantically invalid state when one of its intentions can’t be realized as a result of the state. When the environment provides an input into this closed world, in order to continue successfully realizing its intentions, the design must prevent semantically

invalid states from persisting in its reality. To achieve this, semantic invariants act as markers which identify the semantically invalid state and predict the failure of downstream behaviors. This establishes an invariant path from the semantically invalid state to the intention that can't be realized and in order to restore semantic validity, the design must provide a semantically valid path to resolve the state.

Since the design is completely unambiguous and semantic in nature, another valid way to represent it as potential narratives within a closed semantic world and I will be using this framing in the rest of this document because it is more accessible. These narratives can be constructed by identifying the participants involved, the transformation being applied and the decisions that were made. When an environment provides an impulse into the design, a specific narrative is selected in this world. When looked at in this way, a semantically invalid path can be seen as a narrative that must be eliminated from the potential narratives of this closed semantic world and instead, the design must provide a semantically valid narrative for the environmental impulse as it moves through the design. By only allowing semantically valid narratives in the closed semantic world, the design eliminates failures with respect to all the known invariants.



In the first path, since the design does not provide a semantically valid narrative for the environment, the protected behavior is reached. The failure can be automatically debugged because the failure will be predicted by the violated invariant. However, in the second path, the design does provide a semantically valid narrative to restore semantic validity and the potential bug is eliminated. Using this structure, given a semantically invalid state, the invariant can be automatically tested by walking the path and identifying if the design provides a semantically valid narrative. In this sense, it is also fair to say that semantic invariants protect the downstream behavior because they ensure that the protected behavior can never be reached through the semantically invalid narrative.

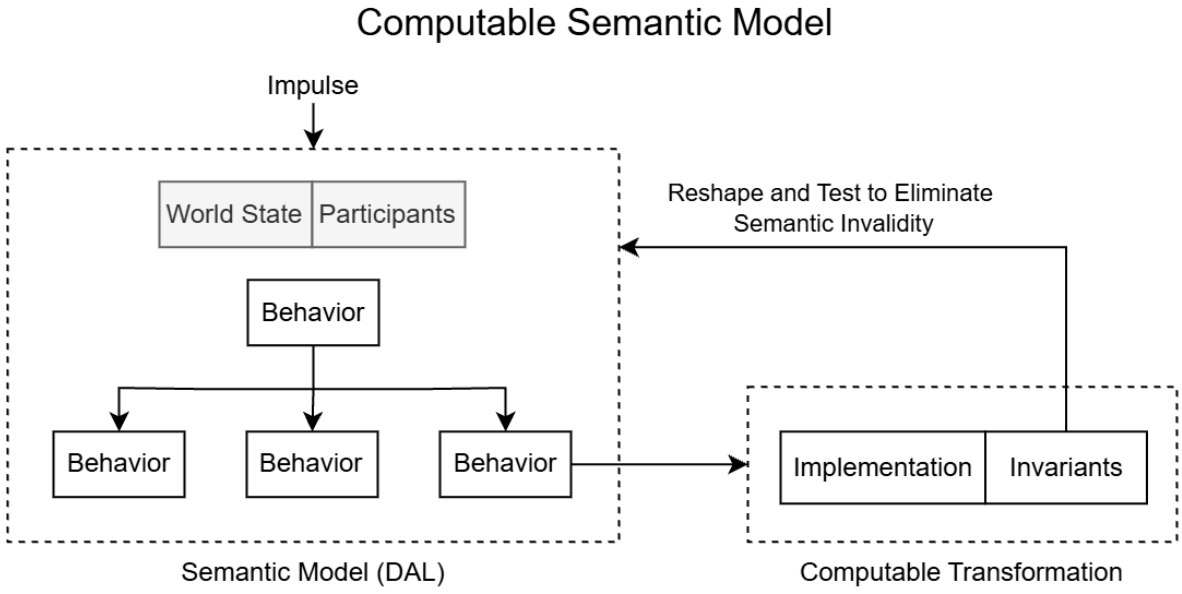
Since a design is a closed semantic world, the invariants intrinsic to the design can be identified through the control flow and data dependencies within the design's structure. As such, the design intrinsic invariants are identified by definition and the behavior of the design can be modified to ensure that it respects every invariant and is internally consistent.

If a design is tested to ensure that it respects all its known invariants and it still fails, it means that the environment which caused the failure is revealing unknown semantics that the design must learn through root cause analysis. This automates failure diagnosis because there is no other interpretation of the failure. In this process, the design becomes progressively more intelligent as it learns new semantics and invariants so that it can continue realizing its intentions in its operational environment.

In this sense, through the invariants, the design absorbs the domain knowledge needed to prevent the failures within its closed semantic world. The design itself is reshaped by the invariants to only allow semantically valid narratives to persist. In this process, by working with the design, debugging, testing and failure diagnosis are automated. However, it is clear that the diagnostic automation isn't a result of complex analysis of the world, instead, it is the result of the world being fully constructed, resulting in unambiguous diagnosis of failures. In this sense, every time ambiguity is eliminated in a meaningful way in the closed semantic world, a new form of automation will emerge.

Computable Semantic Model

Software systems are the result of an intentional design.The design is realized through an implementation in a programming language, which uses its abstractions to realize the design's intentions. Therefore, the meaning behind the mechanical implementation in a programming language is established by the design of the system. Through this process of understanding the execution through the design, the automation enabled by the design is inherited by any diagnostic tool for software systems.



This is achieved by establishing the design as a Computable Semantic Model (CSM) in a Design Abstraction Language (DAL). A CSM is built by establishing the design structure and its transformations unambiguously in the DAL and then defining the implementation which realizes

the meaning established by the design. This effectively means that the implementation exists in the context of its role in the design and any invariants specified by the implementation can be enforced by the design.

This process of establishing the design and its implementation through the CSM means that the invariants established by the implementation of the transformation can reshape the permitted narratives in the world to prevent failures. It also provides a means to automatically test that the design's behavior respects the invariants specified by the transformation, effectively eliminating known failures by construction.

Before continuing with the larger implications establishing software systems are computable semantic models, I will briefly discuss the Design Abstraction Language and how it enables the computable semantic model to be built. I will also describe how the invariants can reshape the narratives of the closed semantic world.

Design Abstraction Language (DAL)

The Design Abstraction Language (DAL) is a declarative language that enables the specification of closed semantic worlds. The scope of language is determined by any mechanism needed to faithfully represent the design such as the behaviors, participants, control flow and invariants.

An example of a script written in DAL is provided below and while this is a very simple design, it does contain enough functionality to demonstrate how the language establishes a design. In the next section, it will be used to demonstrate how the invariants specified in the implementation of the semantics can be used to reshape the behavior of the design to eliminate semantically invalid narratives.

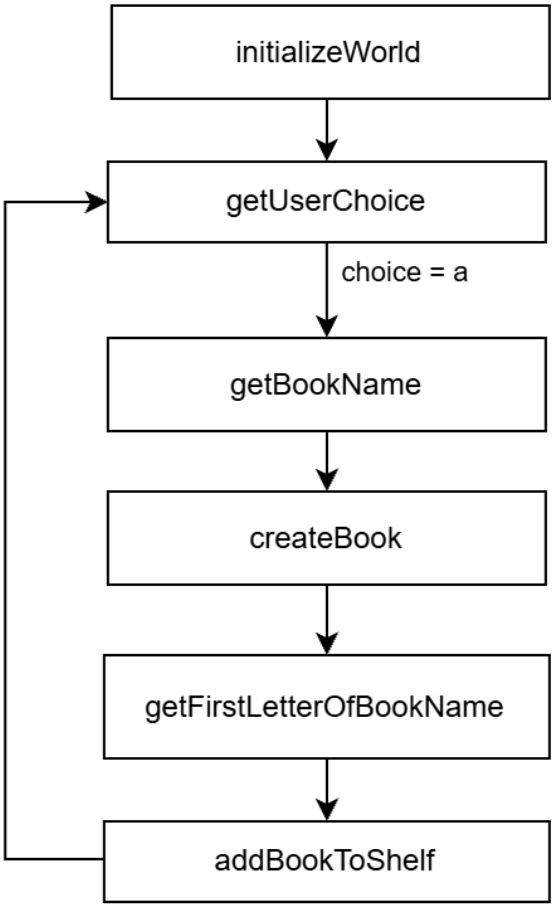
While this is a very simple design, it does contain enough functionality to demonstrate how the language establishes a design. In the next section, it will be used to demonstrate how the invariants specified in the implementation of the semantics can be used to reshape the behavior of the design to eliminate semantically invalid narratives.

```
design("library_manager")

behavior initializeWorld {
  # Initializes the world state with the empty shelf
  shelf = createShelf()
  addToWorldState(shelf , role="shelf ")
  select {
    goToBehavior(getBookName)
  }
}

behavior getUserChoice{
  # Gets the book name
  choice = getInput("Select action (a for add, else exit):")
  addToWorldState(choice , role="choice ")
  select {
    if (choice == "a") {
      goToBehavior(getBookName )
    }
  }
}

behavior getBookName {
  # Gets the book name
  name = getInput()
  addToWorldState(name, role="book_name")
  select {
    goToBehavior(createBook)
  }
}

behavior createBook {
  # Creates the book using the name
  getFromWorldState(name, role="book_name")
  book = createBook(name)
  addToWorldState(book, role="book")
  select {
    goToBehavior(addBookToBasket)
  }
}

behavior getFirstLetterOfBookName {
  # Gets first letter of book name for shelving
  getFromWorldState(book, role="book")
  firstLetter = getFirstLetter(book["name"])
  addToWorldState(firstLetter , role="firstLetter")
  select {
    goToBehavior(addBookToShelf)
  }
}

behavior addBookToShelf{
  # Adds book to shelf using first letter of book name
  getFromWorldState(firstLetter, role="firstLetter")
  getFromWorldState(shelf, role="shelf")
  getFromWorldState(book, role="book")
  shelf= addBookToShelf(shelf, book, firstLetter)
  updateWorldState(shelf, role="shelf")
  removeFromWorldState(firstLetter)
  removeFromWorldState(book)
  select {
    goToBehavior(getBookName)
  }
}
```

Example Design

The behavioral script presented above outlines the design of a simple library manager. In the script, each behavior is established in its own block. A behavior accesses the necessary participants from the world state, performs the transformation on the participants and modifies the world state in an unambiguous way. The behavior then either selects the next behavior directly or selects the next behavior based on the world state.

A diagram of the design established by the script is provided below:



In this example, the closed semantic world has the following participants:

- Shelf
- User Choice
- Book Name
- Book
- First Letter of Book Name

The existence of these participants is established through their behaviors. Initially the world only contains a shelf, then when the user submits a choice, it is now a participant in the world.

Through the choice made, the design selects the next behavior and accepts a book name, resulting in a new participant called name. Then the design creates the book, gets the first letter of the book's name and adds it to the bookshelf.

In this example, there are two points at which the environment provides an impulse into the design. When the user submits their choice and when the user submits the book's name. The design then resolves the input until it reaches a semantically stable state. In this case, once the choice is made and the book's name is provided, a series of behaviors are exhibited in sequence until it is stable again at getUserChoice and waits for the next impulse.

At this point, this structure establishes the semantics of what this world means. In this world getBookName has a specific meaning and it modifies the world state in an unambiguous way. Through the design abstraction language, meaning can be compressed by building composite behaviors. A composite behavior's meaning is established by the behaviors which compose its meaning. For example, if you were to build a composite behavior for adding a book to the shelf, it would contain these behaviors:

- CreateBookAndAddToShelf
    - CreateBook
    - GetFirstLetterOfBookname
    - AddBookToShelf

In this example, the meaning of CreateBookAndAddToShelf is unambiguous. It is a compression of the behaviors that define its meaning. This is no different than how humans use words, when I say the word lion, it has a specific meaning, I have compressed the representation of a lion into a single word.

In this way, the design abstraction language can establish behaviors and compress those behaviors into a composite behavior. The composite behaviors are part of their own semantic world at their level of meaning. As a result, this establishes coherent nested worlds at different levels of meaning. I will speak more about this when I discuss the implications of this framework for distributed systems.

Next, I will talk about how the implementation that realizes the meaning of the semantics is specified.

Implementation of Semantics

The design abstraction language establishes the closed semantic world and the meaning of its semantics. By providing the implementation that realizes the meaning of the semantics, the design itself becomes executable.

Below is an example of the implementation of the transformation that gets the first letter of the books name:

```
def getFirstLetter(value):
    '''
        Get the first character of the string.
    '''
    return value[0]


def getFirstLetter_invariant_1(type, value):
    '''
        The value must be a string and its length
        has to be greater than 0.
    '''
    if type == "evaluate":
        return not (isinstance(value, str) and len(value) > 0)

    if type == "getInvalidValues":
        return [
            # value is not a string
            [
                1
            ],

            # value is an empty string
            [
                ""
            ]
        ]
```

In this example, the implementation realizes the meaning of the transformation getFirstLetter in the getFirstLetterOfBookName behavior of the library manager design. It accesses the first character of the provided value and returns it. The returned value is then saved in the world state as the first letter of the book's name. In my opinion, the important point to note here is that every single part of this has unambiguous meaning, including the participants involved in the transformation, the transformation itself and then the generated value.

Through this process, a software system can be implemented by establishing a design, then the implementation that realizes its meaning and then synthesizing an executable output of the design. The synthesis uses the run command specified in the library manager design to identify which behavior to exhibit first. Then the design itself selects the next behavior to exhibit until it doesn't select a new behavior. As a result, the design itself becomes executable. The closed semantic world is being realized by the implementations that realizes the meaning of the closed semantic world.

Since the implementation of each transformation is realizing the meaning established by the semantics. The ways in which that transformation can fail can also be specified unambiguously as invariants. I will speak more about this in the next section.

Finally, since the design is now a computable model, the minimal information needed to replay an execution can be unambiguously identified. This means that the information that can't be deterministically reproduced by the design must be logged and the replay can recreate the deterministic information through the implementation of the transformations. In the case of the

library manager, this means that the user choice and the book name must be provided, the same execution can be replayed with just those inputs.

Ultimately, this ability to establish the implementation that unambiguously realizes the meaning of the design's semantics transforms the design specified in the DAL into a CSM. Now the implementation unambiguously realizes the meaning of the design and is executed by the design itself.

In the next section, I will talk about how the invariants specified for the implementation of the transformation will reshape the behavior of the design to eliminate semantically invalid narratives.

Reshaping Designs Using Invariants

In the previous section, I talked about how the implementation that realizes the meaning of the semantics is specified. In addition to establishing the implementation, the invariants which define the conditions under which the transformation will fail can also be specified. This means that given particular participant states, this transformation will not be successful.

Since the design is an unambiguous structure, the semantic invariants can be placed at places in the design where the design will inevitably attempt the transformation under semantically invalid conditions. This can be done by identifying when the participant was last updated on any unique path leading to the transformation and placing the invariant at that location. This means that if the invariant is violated, the design must provide a semantically valid path to restore semantic validity and prevent the known failure.

Using the narrative approach to define the world is a much more effective way to communicate this. An invariant identifies a state which will cause a particular narrative to become semantically invalid because its intentions can't be realized. The resolution is that the design must provide a semantically valid narrative to restore the semantic validity.

Invariants don't just have to include a single participant, they can also be for combination of participants. For example, when accessing an entry from the list, a multiple participant invariant is that the accessed position is within the range of the lists length. In this case, along each unique path, the last value of the two participants in the invariant that was updated determines where the invariant gets placed. This is where the world has the potential to become semantically invalid.

To generalize the invariant placement algorithm:

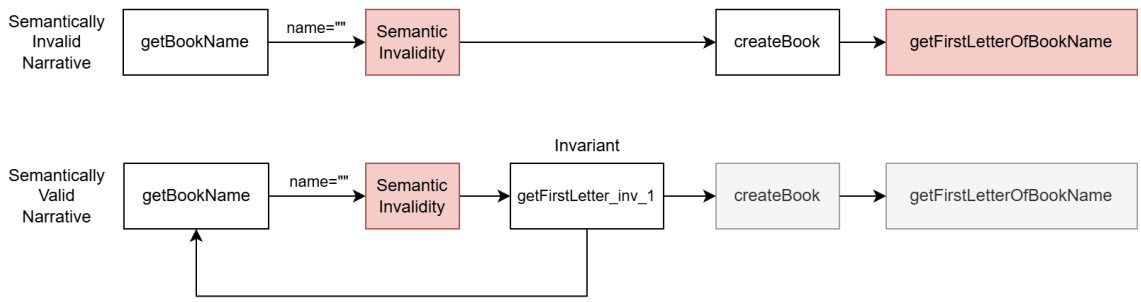
- Identify every unique path leading to the transformation from when the participants in the invariants entered the world state.
- Identify the positions in that path where the participants in the invariant are updated.

- Place the invariant at the last position where one of the participants in the invariant was updated.

In this way, every known semantically invalid narrative will be eliminated from the world because the invariant unambiguously identifies it. A more general way to think about this is that through the invariants, the potential failures are absorbed into the domain knowledge of the world and the known failures are eliminated at the behavioral level by making it impossible for the environment to select a semantically invalid narrative.

Let's apply this to the library manager to identify where the invariant for the getFirstLetter transformation should be placed.

- From the design, it is unambiguous that the participant in the transformation is the participant with the role of "book_name".
- Next, the point at which book_name enters the world state is identified and in this case, that is in behavior getBookName because that is when the participant was added to the world state. If the participant was already in the world state, it would get modified with updateWorldState.
- Next, every unique path from getBookName to getFirstLetterofBookName is identified and in this case, there is only one.
- The invariant is placed after getBookName.



The diagram above visually captures the invariant placement and the path through which the design restores semantic validity. This same process repeats for every path leading to the transformation and is also applied for invariants containing multiple participants. In doing so, every semantically invalid narrative is detected and can be eliminated.

This then sets up the next step where every invariant path can be automatically tested to verify that it provides a semantically valid narrative. This is done by verifying that there is a control flow which evaluates the semantically invalid state and selects a different path than the invalid narratives path. You can also unambiguously identify the invariant directly because it was automatically placed there but testing by actually walking the path is more complete.

Finally, as a result of every invariant being automatically placed and tested to verify that it restores semantic validity, any observed failures must mean that another narrative in this world is semantically invalid and through root cause analysis using the environment that caused the

failure, the design learns new semantics and invariants to eliminate the semantically invalid narrative from the world.

Shared Meaning and Distributed Systems

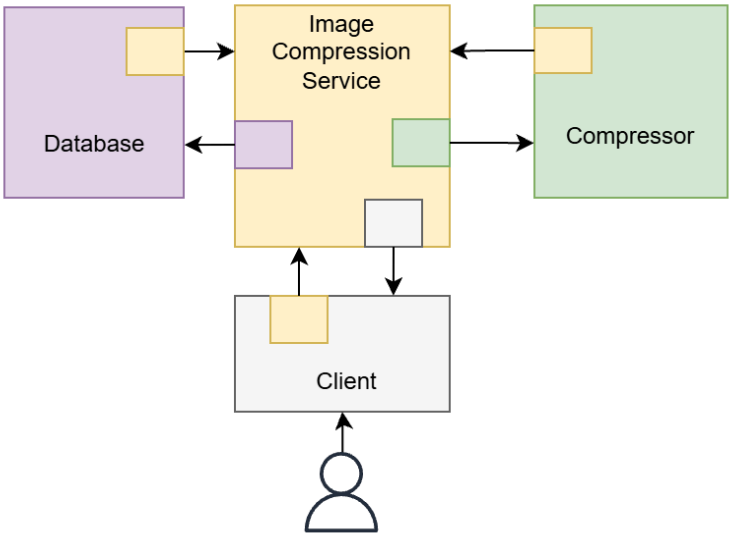
One of the consequences of the computable semantic model is that the meaning behind the execution of a software system is unambiguous. As a result, software systems can achieve successful coordination through shared meaning. I will first talk about how shared meaning leads to successful interaction in a general sense and then apply it to software systems.

Imagine you visit a library, find the book you are looking for and walk up to the librarian with the intention of checking out a book. In order to check out the book, you present the librarian with the book but the librarian was expecting a library card and the interaction halts. In this case, what it means to check out a book is different for the librarian and the visitor. Since there is no shared meaning between the two, the interaction fails and neither the librarian nor the visitor achieve their goal.

In this example, since it is two humans interacting, they can clarify and establish a shared meaning and move forward with the interaction. However, software systems can't improvise(yet), so shared meaning has to be established by construction. The CSM provides the mechanism to eliminate ambiguity in the meaning of interactions between software systems through their design.

Semantically Compatible Designs

Consider the system shown below. This example establishes a distributed image compression service that connects multiple designs together. In this example, each node is its own design. However, when two designs are interacting with each other, in order to successfully coordinate they must share the same meaning.



When the meaning of the database is established, it also establishes the meaning of a database user. Therefore when the image compression service is interacting with the database, it takes on the role of the database user and communicates with shared meaning. In this sense, the interaction between the two designs is semantically valid because they share the same meaning. In the diagram, I highlight this using a color scheme to indicate that each design is invoking shared meaning between both designs.

As a result, through semantically compatible interactions between designs, larger semantic worlds can be constructed. The larger semantic world establishes a new level of meaning that is defined by the semantically compatible designs that make up its reality. This frames a distributed system as a semantic world constructed from semantically compatible interactions between designs.

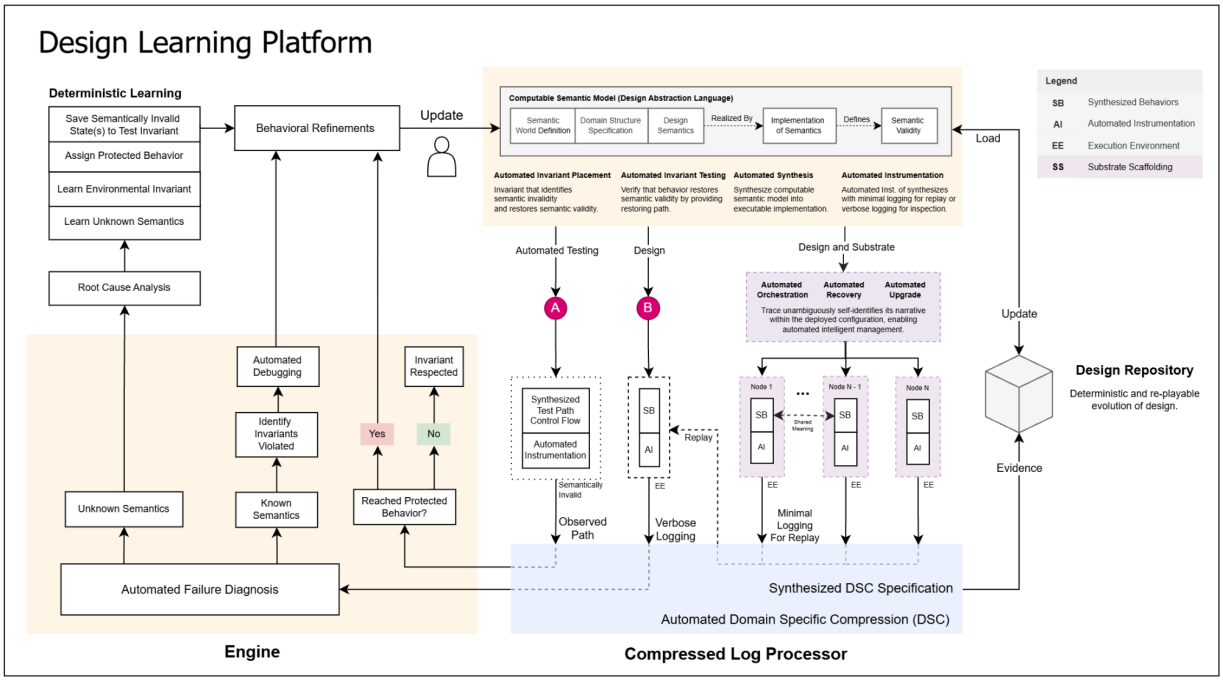
In order for successful interaction, the meaning that must be established on both sides of the interaction is unambiguous and this happens by design. In addition, the same principles that define the correctness of a design are equally as valid at this level of meaning. By tracking interactions between designs, environmental invariants can be identified at a particular level of meaning in the closed semantic world that represents a distributed system.

Securing Interactions Between and Within Designs

Finally, as a result, interaction between designs now has unambiguous meaning. There are meaningful implications for securing systems because the intention of the interaction is unambiguous. While before, the mechanical sequence had to be correct, now the design can ensure that it is the right narrative that is communicating with it. If the design is looked at as the brain of the software system, then this ensures that the correct brain is speaking with, making it more difficult to compromise a system.

There are many approaches to implement this but one could be that the DAL can be extended to generate a dynamic checksum along the narrative path to identify unique narrative identity. There are more interesting ways that I can think of securing it further by sharing the identity of the instance of the brain and using that to generate the proof. Regardless, the point here is, by eliminating ambiguity in a meaningful way, more automation will be enabled and securing the interaction between and within designs is another application for this framework.

Design Learning Platform



In this section I will discuss the Design Learning Platform as the practical framework that realizes this solution. I will also discuss how the practical hurdles faced when implementing this solution at scale are overcome.

The CSM is the mechanism through which ambiguity is eliminated in the construction of software systems. This means an engine that operates on the CSM can automatically generate the information needed for it to unambiguously understand the designs execution and enable deterministic learning. The engine does not analyze or arrive at conclusions on its own, instead, it simply enforces the correctness established by the model.

As mentioned earlier, every time that ambiguity is meaningfully eliminated from the CSM, a new form of automation emerges. In this document, I've described how the following is automated:

- Failure Diagnosis: Known invariant violation or unknown invariant is identified.
- Debugging: Which invariant was violated?
- Testing: Does the design respect all known invariants?
- Documentation: Design written in the DAL represents itself.

Now I will talk about how the engine practically achieves the failure diagnosis, debugging and testing.

The engine uses the invariants specified for the computable transformation to automatically place the invariants using the design's structure. It then uses the invariants definition to establish a semantically invalid state, then it walks the invariant path to find the control flow that restores semantic validity and establishes whether it selects a semantically valid narrative. If no such control flow exists, then it concludes that the design behavior does not respect the invariant. If such a control flow does exist, it knows that the semantically invalid narrative is eliminated from the closed semantic world.

After this process, since every known failure is eliminated by construction and verified through the invariant testing, failure diagnosis is automated because an observed failure can only mean that new semantics must be learnt through root cause analysis. To achieve this, the engine automatically instruments the synthesized executable output to log the necessary information needed to replay the failed execution. This is once again possible because of the complete construction of the CSM because it unambiguously identifies all the information that can't be deterministically reproduced. Then the failed execution of the CSM can be replayed using the logged inputs to aid in the root cause analysis.

After root cause analysis and the expansion of the semantics, the same process repeats and the design is tested to ensure that it respects the new invariant. In this way, the engine essentially tests that the design respects every invariant to automate failure diagnosis and enables deterministic learning from failures by enabling root cause analysis on the environment which reveals the new semantics. This means that the environment which reveals the new semantics must be losslessly preserved at scale. If it isn't, then the automatic and deterministic nature of this process is broken.

To address this, the design's unambiguous nature can once again be exploited to provide the domain structure of the participants. Using this structure, domain specific compression can be applied to preserve all the environments at minimal cost. Domain specific compression is a technique that exploits the known domain structure of the data to maximize its compression.

To practically achieve this, an open source tool named Compressed Log Processor (CLP) can be leveraged. It is a tool that can apply domain specific compression to the data and also search the compressed data without decompression. More importantly, it has been proven at a petabyte scale, establishing that it is possible to preserve the environments to learn new semantics through root cause analysis at scale.

Since every environment that motivated new semantics is preserved, it results in a design repository through which the evolution of the software system can be deterministically replayed. The repository tracks not just the changes to the implementation but also how the system was designed, executed and refined through learnt semantics over time.

This entire process is captured in a framework called the Design Learning Platform(DLP). It leverages the CSM and domain specific compression to fully automate the diagnosis of software

failures and deterministically learn from losslessly preserved environments. In the process, it automates failure diagnosis, debugging and testing.

In the next section, I will talk about how the CSM enables automated orchestration of software systems, including deployment, recovery, upgrade and lifecycle management. I will discuss how, when combined with deterministic learning enabled by automated failure diagnosis and testing, this framework fully automates the management of software systems.

Automating Software Systems Management

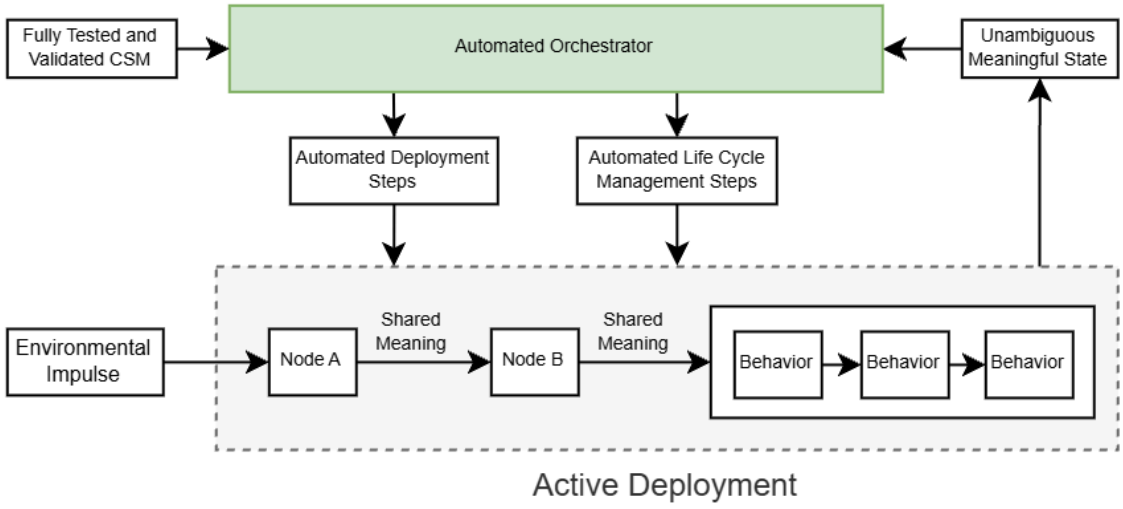
In order for a design to successfully realize its intentions, the conditions necessary for it to be able to realize its intentions must exist. This can be as simple as deploying the deigns onto a node, performing life cycle operations or more specific tasks like recovering from failures. As such, when the design of a distributed system is established, it also defines a manager that can successfully orchestrate the distributed system.

Orchestrating is ultimately responding to the state of the system appropriately to restore the system's ability to realize its intentions. Like with all other processes in this framework, the key to automating it is to eliminate ambiguity. Through the CSM, the meaning of a software systems state is entirely unambiguous. Each impulse into the system creates an unambiguous narrative that ripples through the system, even if it ends in failure. Since the narrative is un ambiguous, there can be an automatic unambiguous response by the management system.

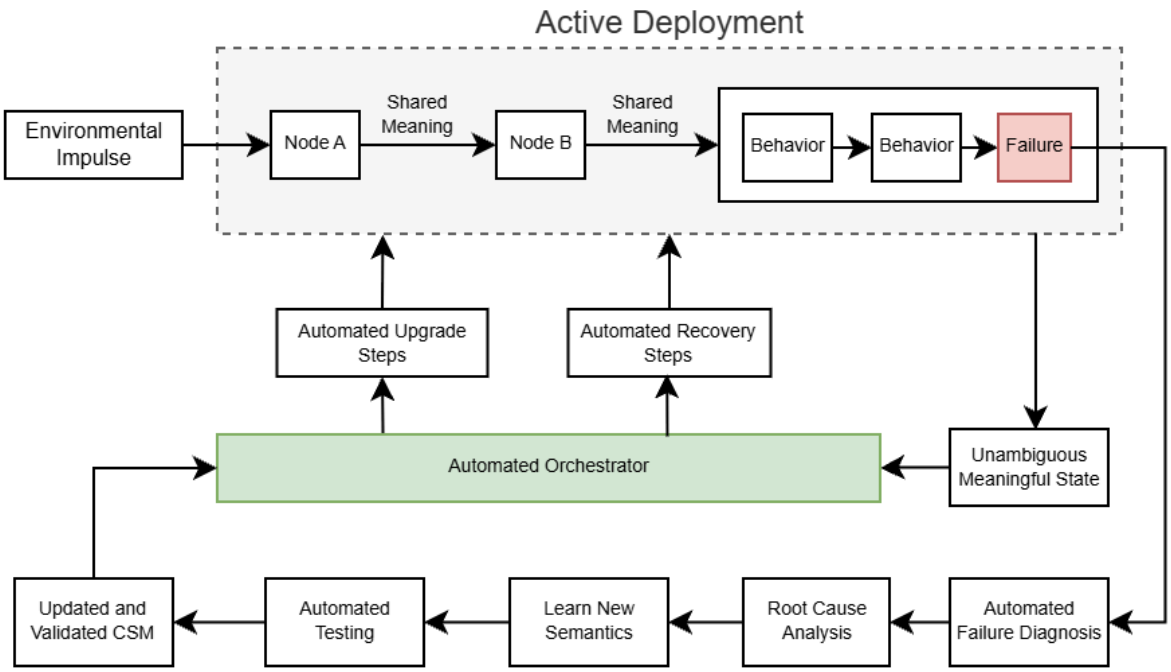
Once again, this is made possible by eliminating the ambiguity in the meaning of software systems. In this case, as the impulse moves through the system, it can self-identify the narrative that it is executing, for example:

"At 9:01 AM I started when the user chose to add a book to the library. Then they submitted a book name and then I created the book. However, I failed when I attempted to read the first letter of the book's name."

The failure in this narrative won't actually happen because the invariants in the CSM would have identified it and testing would have eliminated it. However, it is a simple example to convey the idea and this extends to distributed systems naturally because the designs have shared meaning. While recovering from failures is one part of the orchestration, the same principle applies to life cycle management because the meaning of the system's state is unambiguous and the response to it is unambiguous.



This means that the manager must be designed to respond appropriately to every meaningful state in the design it is managing. Practically, it won't have to account for every narrative, instead, each design can self identify the states that will need to be managed and internally group narratives into meaningful states. Through this, an enumeration of every meaningful state can be obtained from the design and the manager can be tested to ensure that it responds appropriately.



When a failure occurs, automated failure diagnosis is performed and the diagnostic data collected from the automated instrumentation is presented to the developer for root cause analysis to deterministically learn new semantics and prevent future failures. Once the design learns new semantics and it is automatically tested and validated, the automated orchestrator can upgrade the design with the updated CSM.

At the same time, to immediately respond to the failure and restore the system, the orchestrator automatically responds to the state of the system and takes unambiguous steps to restore the system's ability to realize its intentions.

The intelligence of the response by the orchestrator is entirely up to its design, since there is no ambiguity in the state it is responding to or the design it is managing, it can surgically recover the system to restore normal functionality or upgrade the system to deploy the updated CSM. Ultimately, when combined with the deterministic learning loop established in earlier sections, the ability to automatically orchestrate the software system fully automates the management of software systems.

Conclusion

The techniques presented in this document take a fundamentally different approach to software systems management than existing approaches. It provides the means to complete the construction of a software system by establishing its design in a Design Abstraction Language as a Computable Semantic Model that unambiguously establishes its meaning.

This results in the complete automation of software systems management because there is no ambiguity in the meaning of the system's state and the necessary response to restore its ability to realize its intentions. This includes recovering from failures, deployment, normal life cycle operations and upgrades.

In addition, through the CSM, the diagnosis of software failure is automated and it results in a design learning platform that enables the design to learn new semantics through root cause analysis on unambiguous environments that caused the failure. To enable the automation at scale, it leverages the unambiguous nature of the CSM to preserve the environments which reveal new semantics using domain specific compression.

Ultimately, this solution does not invent anything new. Software systems have always operated at this level of meaning and there is nothing that this solution presents that wasn't already common knowledge. What makes it novel is that it takes steps to unambiguously understand the implementation of the software system through the meaning established by its design. It achieves this using the computable semantic model and the ambiguity that is eliminated enables the management of software systems to be fully automated. It treats any manual effort as a symptom of incomplete construction and provides the means to complete the construction through the CSM.

This solution ultimately eliminates ambiguity in the development and maintenance of software systems. As a result, the software systems that run modern society are less likely to fail and if they do fail, an automated process ensures their recovery and automated failure diagnosis supports deterministic learning to prevent future failures. The future of software development will be less about writing bug free code and more about building intelligent and secure systems that understand each other.